第一章 Agent, 工作流和 Harness

1.1 谁决定下一步：工作流与 Agent

$$Agent = LLM + 上下文 + 工具$$

LLM 是大脑，负责做决策；上下文是眼睛，决定模型这一步能看到什么；工具是手脚，用来查询或者改变外部世界.

Agent& 工作流

Context、Tools、Constrain、Verify、Correct 代码写死步骤，就是工作流 (Workflow)；由 LLM 在运行时决定下一步，就是 Agent。

两者都是 LLM 加其他组件组成的系统（compound AI system），区别只在流程由谁决定。

用代码解释:

1. 工作流：控制流写在代码里，LLM 只是某个函数内部的一次调用。

Listing 1.1: 工作流

```
def book_flight(req):
  user = verify_identity(req)
  flights = search_flights(req) # 这里面可以调 LLM
  pay(user, flights[0]) # “付款前不能预订”由代码保证
  return confirm(user)
```

2.Agent: 控制流变成了模型在运行时吐出来的数据。代码只剩一个循环，每一圈都问模型：“下一步做什么?”

Listing 1.2: agent

```
while True:
  action = llm(context) # 由模型决定下一步
  if action.is_final:
     return action.answer
  obs = execute(action) # 程序真正执行这个动作
  context.append(action, obs) # 执行结果进入下一轮上下文
```

1.2 工具调用怎么发生

模型从来不执行工具，它只“说”自己想调用哪个工具、传什么参数。真正去执行的是你的程序。

模型调用的四步流程： 1. 声明工具 2. 模型决定调用 3. 结果追加上下文 4. 模型基于结果回复

代码举例：以查天气为例子

第一步：声明工具你的程序发出第一次请求，除了消息列表，还要在单独的 tools 字段里告诉模型有哪些工具可用：

```
 {
 "messages": [
   {"role": "system", "content": "你是天气助手，需要实时数据时调用工具。"},
   {"role": "user", "content": "温哥华现在多少度？"}
 ],
 "tools": [{
   "name": "get_weather",
   "description": "查询城市的实时天气",
   "parameters": {"type": "object",
                "properties": {"city": {"type": "string"}},
                "required": ["city"]}
 }]
}
```

第二步：模型决定调用: 模型不回答问题，而是返回一条带 tool_calls 的 assistant 消息，然后这次请求就结束了。模型没法停在那里等结果。

```
{
{"role": "assistant", "content": null,
 "tool_calls": [{"id": "call_1",
               "function": {"name": "get_weather",
                          "arguments": "{\"city\": \"Vancouver\"}"}}]}
}
```

第三步: 程序执行，把结果追加到上下文。代码解析 tool_calls，校验参数，真的去调天气接口，然后往消息列表里追加两条消息：

```
{
{"role": "assistant", "tool_calls": [ ...第   步那条，原样放回... ]},
{"role": "tool", "tool_call_id": "call_1", "content": "{\"temp\": 13, \"conditions\": \"晴
    \"}"}
```

第四步：模型基于结果回复。你的程序发出第二次请求，把完整的消息列表重新发一遍。模型这时才看到结果，回答：“温哥华现在晴，13°C。”

三条规则

1. API 是无状态的。模型不记得上一次请求里发生过什么，每次请求都要把完整历史重新发送。所以第 1 节 Agent 循环里那句 `context.append(...)` 不是可有可无，没有它模型就失忆了。
2. `tool_call_id` 必须配对。一条 assistant 消息里可以同时有好几个工具调用。书里多币种汇总的例子，第一轮就一次发起了 3 次汇率转换。每条 tool 消息要靠 id 说明自己是哪一个调用的结果。
3. 不要把工具结果伪装成 user 消息。模型会以为是用户又问了一个新问题，多步推理就接不上了。

1.3 Harness

Harness 指的是 Agent 内部、模型之外的所有代码，负责让 Agent 跑起来、并且管住它。第 2 节里“解析 `tool_calls`、执行工具、追加消息”的那部分代码，就是 Harness 最基本的一层

> Agent = Model + Harness
> Harness = 上下文管理 + 工具接口 + 约束 + 验证 + 纠正

边界要分清：Harness 不包括环境。沙盒里的文件、数据库、用户、外部网站都属于环境（Environment）。Harness 可以创建或隔离环境，但环境本身不归它。

表 1.1: Harness 五要素

要素	职责与原则	内核里对应什么	例子
Context 上下文	信息要充分	给进程准备好它需要的数据	系统提示词、知识库、历史压缩
Tools 工具	接口要清晰	系统调用表	MCP 工具、代码解释器
Constrain 约束	故障安全，默认关闭	权限检查	危险工具默认要用户批准
Verify 验证	检查结构化数据，不检查自由文本	校验参数和返回值	linter、类型检查、跑测试
Correct 纠正	静默重试，确认无法恢复之前不暴露中间状态	错误处理和恢复	重试、熔断、转人工

下面是控制循环的简易代码

Listing 1.3: 控制循环

```
step = 0
while step < MAX_STEPS: # 停止条件，防止死循环
  action = llm(build_context(history)) # Context：决定模型这一轮看到什么
  if action.is_final:
      return action.answer
  if not allowed(action): # Constrain：没权限就拦下
      history.append(action, "被拒绝：需要用户批准")
      step += 1
      continue
  obs = execute(action) # Tools：真正执行
  if not verify(obs): # Verify：检查结果对不对
      obs = correct(action, obs) # Correct：重试或者换路
  history.append(action, obs)
  step += 1
```

1.4 agent 工程范式

agent 工程重心演变由提示工程到 Graph 工程

> 提示工程 → 上下文工程 → Harness 工程 → Loop 工程 → Graph 工程

表 1.2: 从提示工程到 Graph 工程

层次	关注什么
提示工程	一次调用的提示词怎么写
上下文工程	每次调用往窗口里放哪些信息
Harness 工程	模型外面的整套运行与治理：约束、验证、纠正
Loop 工程	让循环跨轮次、跨会话持续运转
Graph 工程	把循环、确定性程序和人工审批组织成一张执行图

构建有效 Agent 的三原则：

1. 保持简单：先直接调 API，确实需要时再上框架。
2. 保持透明：把规划、日志和决策轨迹都暴露出来。
3. 设计好 Agent 与计算机之间的接口（ACI）：站在 Agent 的角度设计工具，用防呆（Poka-yoke）的思路，让错误根本没机会发生。ACI 不只是写代码，工具描述也是接口的一部分。
   防呆的核心：与其写提示词提醒模型“请注意不要犯错”，不如改接口，让那个错误写不出来，或者一写出来就得到清楚的报错。

1.5 上下文 React 轨迹

调用 LLM 上下文分为五个部分：

1. 系统提示词
2. 工具定义
3. 用户消息
4. 模型回复
5. 工具执行结果

前两项（系统提示词 + 工具定义）是静态前缀，后三项（用户消息 + 模型回复 + 工具执行结果）是随交互不断增长的动态消息历史。这五个部分共同构成了 LLM 每次推理时的上下文。

$$\text{每次调用模型的上下文} = \text{静态前缀} + \text{轨迹}$$

定义 1.1

React 是 Reasoning+Acting 缩写，每一圈三步：

1. 思考：模型想下一步该做什么
2. 行动：发出工具调用
3. 观察：读取工具结果

♣

转完一圈，轨迹多出一段，然后进入下一圈，直到模型直接给出答案。这就是第 1 节那个 while True 循环在做的事。所以为什么要把思考和行动交替进行?

1. 只思考不行动（纯 CoT）：模型只能凭记忆推理，容易编造事实。
2. 只行动不思考：盲目调用工具，不知道为什么调、调完怎么用。
3. ReAct：每一步的思考都建立在上一步的观察之上，幻觉更少。